
CS 61B

Spring 2025

Midterm 2

Wednesday, April 2, 2025

Your Name: Solutions (last update: Mon Apr 8, 12:00 AM)

Your SID: _____ Location: _____

SID of person to your left: _____ Right: _____

This exam is worth **100 points**, and consists of 6 questions. Questions are ordered by topic, not by difficulty.

Question:	1	2	3	4	5	6	Total
Points:	16	14	31	16	12	11	100

- ☐ indicates only one circle should be filled in. ☐ indicates more than one box may be filled in. **Please fill in the shape completely.** If you change your response, **erase as completely as possible.**
- Anything you write that you ~~cross out~~ will not be graded.
- If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade according to the worst interpretation.
- Unless otherwise specified, all data structures and algorithms behave according to their implementation in lecture, with no additional optimizations.
- If an implementation detail (e.g. tiebreaking scheme, linked list topology) is relevant, it will be explicitly noted in the question.

Coding questions:

- You may write at most one statement per blank and you may not use more blanks than provided.
- Your answer will be reformatted according to the 61B style guidelines. For example, any if-statement requires at least three lines for the purposes of determining line count.
- You may not use ternary operators, lambdas, streams, or multiple assignment.
- Unless otherwise specified, you may assume that everything on the reference card has been imported.

Write the statement below in the same handwriting you will use on the rest of the exam. Failure to do so may result in a voided exam.

I have neither given nor received help on this exam (or quiz), and have rejected any attempt to cheat. If these answers are not my own work, I may be deducted up to 9,876,543,210 points.

Signature: _____

1 Potpourri

(16 points)

- (a) (1 point) True/False: All valid Left-Leaning Red-Black Trees have the same number of black links along every root-to-leaf path.

☒ True☐ False

- (b) (1 point) True/False: If a binary search tree has its edges colored red/black such that the number of black links along every root-to-leaf path is the same, then it is a valid Left-Leaning Red-Black Tree.

☐ True☒ False

- (c) (1 point) True/False: If all edges in an undirected, connected graph have the same weight, any spanning tree of the graph is a valid Minimum Spanning Tree.

☒ True☐ False

- (d) (1 point) True/False: While running Kruskal's algorithm, we multiply all edge weights of an undirected, connected graph by -1 before inserting them into the heap. The resulting spanning tree will be a **Maximum** Spanning Tree of the original graph.

☒ True☐ False

- (e) (1 point) True/False: Let G be an undirected, connected graph with a unique Minimum Spanning Tree, and let C be a cut of G with more than one crossing edge. The edge with the maximum weight across C is never included in the resulting Minimum Spanning Tree.

☐ True☒ False

- (f) (1 point) True/False: In Quick Find, the isConnected operation has a runtime of $O(1)$.

☒ True☐ False

- (g) (1 point) What is the fewest number of edges in an undirected graph with N vertices and exactly 3 connected components?

☐ 1 edge☐ $N - 1$ edges☐ 2 edges☐ $N - 2$ edges☐ 3 edges☒ $N - 3$ edges☐ 4 edges☐ None of the above

For parts h and i, consider a modified heap, where instead of using a resizing array to store elements, we use a circular doubly-linked list with a sentinel.

- (h) (2 points) What is the best-case runtime for `removeMin` in this heap, assuming the heap contains N elements?

- ☒ $\Theta(1)$
☐ $\Theta(N \log(N))$
☐ $\Theta(\log(N))$
☐ $\Theta(N^2)$
☐ $\Theta(N)$
☐ None of the Above

- (i) (2 points) What is the best-case runtime for `insert` in this heap, assuming the heap contains N elements?

- ☐ $\Theta(1)$
☐ $\Theta(N \log(N))$
☐ $\Theta(\log(N))$
☐ $\Theta(N^2)$
☒ $\Theta(N)$
☐ None of the Above

- (j) (2 points) Which of the following is true about the array, `[-, 0, 0, 2, 5, 0, 7, 5]`? Select all that apply.

- ☐ The array is a valid backing array of a max-heap.
☒ The array is a valid backing array of a min-heap.
☒ If the `-` is replaced with `-8`, the array is a valid backing array of a Weighted Quick Union **without** path compression.
☒ If the `-` is replaced with `-8`, the array is a valid backing array of a Weighted Quick Union **with** path compression.
☐ None of the above

- (k) (2 points) Suppose a binary search tree contains the following elements: `{1, 2, 3, 4, 5, 6, 7, 8}`, with the root containing 7. After deleting the root node using Hibbard deletion, which of the following nodes could possibly be the new root of the tree? Select all that apply.

- ☐ 1
 ☐ 5
☐ 2
 ☒ 6
☐ 3
 ☒ 8
☐ 4
 ☐ None of the Above

- (l) (1 point) Given the backing array `parents` of a Weighted Quick Union, which of these returns the size of the largest connected component?

- ☐ `min(parents)`
☐ `max(parents)`
☐ `max(parents) * -1`
☒ `min(parents) * -1`
☐ None of the above

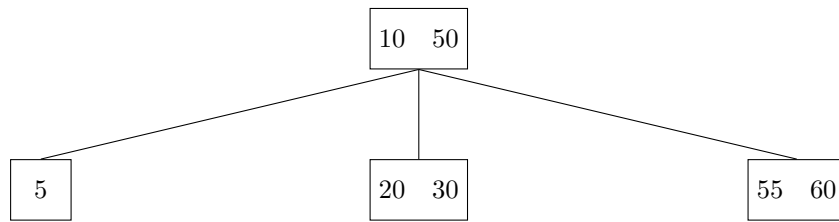
Solution:

- (a) This is due to the one-to-one correspondence with B-trees
- (b) A simple example is a (big) tree where every edge is colored red; all root-to-leaf paths have 0 black edges, but it's not a valid LLRB
- (c) Any spanning tree over a graph with N nodes needs exactly $N - 1$ edges, and will thus have the same weight as any other spanning tree.
- (d) Kruskal's will instead take the largest edges first, which would fulfill the maximum version of the cut property.
- (e) Consider a tree with any cut crossing multiple edges. The minimum spanning tree will use all edges, and therefore the maximum-weight edge in C .
- (f) We just check the values of the sets and check if they are equal.
- (g) Each edge connects at most two connected components together, so we need at least $N - 3$ edges before the number of connected components is 3.
- (h) Move the last item to the root, and in the best case, it is already less than or equal to the two children.
- (i) After we add the item to the end, we need to check if it's less than its parent. It takes $\Theta(N)$ time to find that parent (since the item is at index $N/2$), and in the best case the parent is less and we don't need to go further.
- (j) Each one can be evaluated separately. Note that all valid path-compressed WQUs are also valid WQUs without path compression, since we can always connect roots and not have any path compression.
- (k) The only items that can replace the root without moving any other items in a binary tree are the ones immediately before and after the current item (regardless of the tree structure)
- (l) The size of connected components are stored as negative numbers in the array, and we want the smallest such negative number for the largest component. After that, we multiply by -1 to get a positive value.

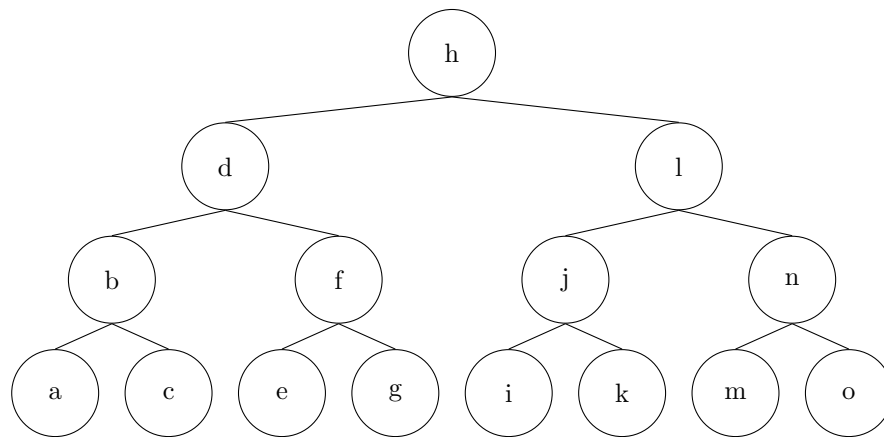
2 LLRB

(14 points)

(a) (5 points) Convert the 2-3 tree below into its corresponding Left-Leaning Red-Black Tree (LLRB).



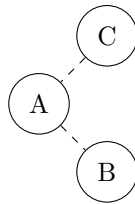
Select the value that goes in each node of the LLRB below and whether or not the node is red. If a node is unused, select “None”. As a reminder, a node is red when the edge to its parent is red, and the root is never red.



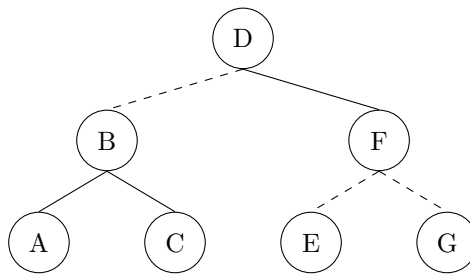
	5	10	20	30	50	55	60	None	isRed
a	☐	☐	☐	☐	☐	☐	☐	☒	☐
b	☒	☐	☐	☐	☐	☐	☐	☐	☐
c	☐	☐	☐	☐	☐	☐	☐	☒	☐
d	☐	☒	☐	☐	☐	☐	☐	☐	☒
e	☐	☐	☒	☐	☐	☐	☐	☐	☒
f	☐	☐	☐	☒	☐	☐	☐	☐	☐
g	☐	☐	☐	☐	☐	☐	☐	☒	☐
h	☐	☐	☐	☐	☒	☐	☐	☐	☐
i	☐	☐	☐	☐	☐	☐	☐	☒	☐
j	☐	☐	☐	☐	☐	☒	☐	☐	☒
k	☐	☐	☐	☐	☐	☐	☐	☒	☐
l	☐	☐	☐	☐	☐	☐	☒	☐	☐
m	☐	☐	☐	☐	☐	☐	☐	☒	☐
n	☐	☐	☐	☐	☐	☐	☐	☒	☐
o	☐	☐	☐	☐	☐	☐	☐	☒	☐

The trees below represent LLRBs immediately after a new element has been added, but before any invariant-fixing operations have been taken (dotted edges represent red edges and solid edges represent black edges). For each tree, write out the sequence of operations (ex. `rotateLeft(A)`, `rotateRight(B)`, `colorFlip(C)`) needed to fix all invariants. If it is impossible to have created the LLRB by adding a single new element, write “Not possible”. Note that the letters are simply placeholders.

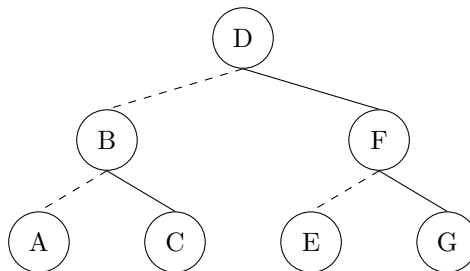
(b) (3 points)



(c) (3 points)



(d) (3 points)



Solution:

- (a) See chart above.
- (b)
 - 1. RotateLeft(A)
 - 2. RotateRight(C)
 - 3. ColorFlip(B)
- (c)
 - 1. ColorFlip(F)
 - 2. ColorFlip(D)
- (d) Not possible. There are root-to-null paths of lengths 1, 2, and 3, which should not be possible immediately after insertion.

3 Anirudh's Weighted Question

(31 points)

Anirudh the Adventurer wants to find a treasure hidden on one of N islands. Luckily, he has discovered the secret library of Stella the Sneaky Sorceress. The library contains many scrolls that each represent a map of the N islands. On each scroll, there is an array of N numbers. Each array is a representation of a Quick Union tree with a single root. For example, for $N = 5$, one possible array might be $[1, 2, 3, -5, 0]$.

Anirudh knows that all of the Quick Union trees are valid, but only one of them is a valid Weighted Quick Union (without path compression). The one scroll with a valid Weighted Quick Union is the only real map. By “valid WQU”, we mean that the given array can be created by a sequence of union operations (under some tie-breaking scheme). For example, $[1, 2, 3, -5, 0]$ is a valid QU, but not a valid WQU.

To determine which map is real, he will first convert each array into a `TreasureMap`, which is a linked data structure rather than an array. Note: Anirudh's nested `Island` class represents a node of the tree, and each node keeps track of its children. Then he will call `isValid` on the `TreasureMap`. Fill in the functions in the `TreasureMap` class to help Anirudh find the treasure.

```
public class TreasureMap {

    private Island root;

    private class Island {
        public int label;
        public List<Island> children;

        public Island(int label) {
            this.label = label;
            this.children = new ArrayList<>();
        }
    }

    // Converts the underlying array representation of a QuickUnion into a corresponding TreasureMap.
    public TreasureMap(int[] qu) {
        // part a
    }

    // Checks if the TreasureMap is a valid WeightedQuickUnion
    public boolean isValid() {
        // part e
    }

    // Helper function for isValid
    // Returns -1 if parent is not a valid WQU
    // Otherwise returns the number of nodes in the subtree rooted at parent
    private int size(Island parent) {
        // part e
    }
}
```

- (a) (8 points) Complete the constructor of the `TreasureMap` class, which receives as input the backing array of a Quick Union with exactly one root.

```
public TreasureMap(int[] qu) {
    Island[] isles = new Island[qu.length];
    for (int i = 0; i < qu.length; i++) {
        isles[i] = new Island(i);
    }

    for (int i = 0; i < qu.length; i++) {
        int parent = qu[i];

        if (parent < 0) {

            root = isles[i];
        } else {

            isles[parent].children.add(isles[i]);
        }
    }
}
```

- (b) (4 points) What are the best-case and worst-case runtimes of this function? Express your answer as a Θ bound in terms of N , the number of islands.

Best-Case Runtime:

 $\Theta(\quad)$

Worst-Case Runtime:

 $\Theta(\quad)$

Solution: Best-Case Runtime:

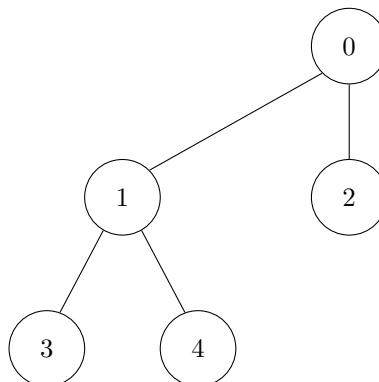
 $\Theta(N)$

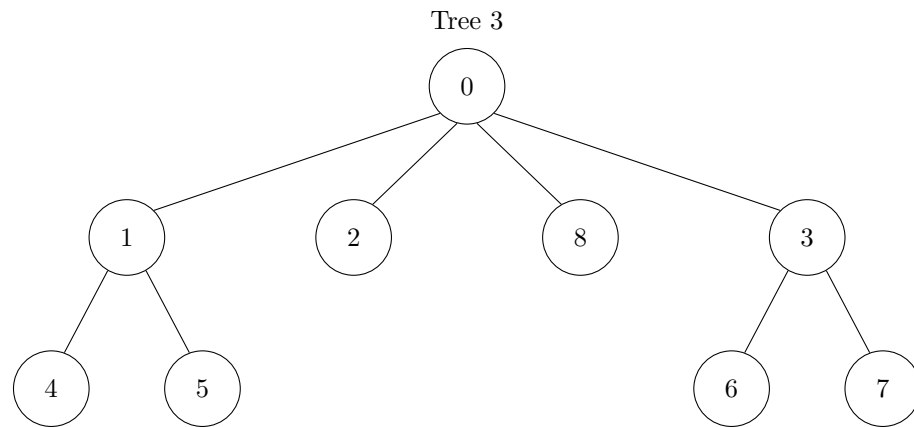
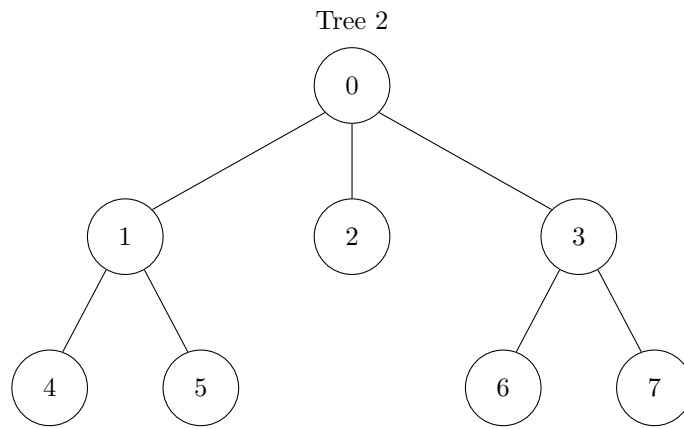
Worst-Case Runtime:

 $\Theta(N)$

- (c) (2 points) In order to gain some intuition behind the problem, Anirudh decides to consider three small test cases to see what properties valid Weighted Quick Unions (WQUs) have. **Two of the trees below are invalid, and one is valid.** Determine which tree is valid. The next two trees are on the following page.

Tree 1

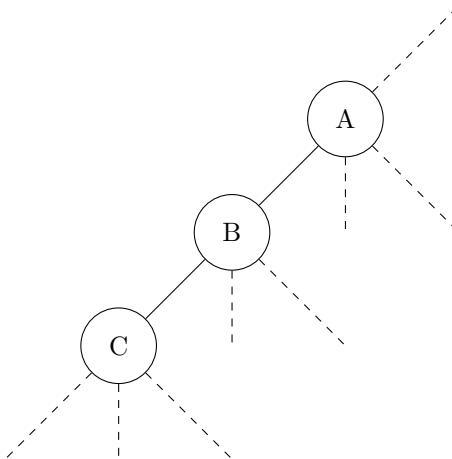



☐ Tree 1 is valid

☐ Tree 2 is valid

☒ Tree 3 is valid

- (d) (1 point) True/False: Consider a WQU where node A is a parent of node B, and node B is a parent of node C. Node C must have been connected to node B before node B was connected to node A.


☒ True

☐ False

- (e) (16 points) Complete the `isValid` method, which determines if the `TreasureMap` corresponds to a valid Weighted Quick Union, and the `size` helper method. Your solution must run in $O(N \log N)$ time, where N is the number of islands.

Warning: We consider this question to be significantly harder than the other questions on the exam.

```
// Checks if the TreasureMap is a valid WeightedQuickUnion
public boolean isValid() {

    return size(root) != -1;
}

// Helper function for isValid
// Returns -1 if parent is not a valid WQU
// Otherwise returns the number of nodes in the subtree rooted at parent
private int size(Island parent) {

    int currSize = 1;

    PriorityQueue<Integer> childSizes = new PriorityQueue<>();

    for (Island child : parent.children) {

        childSizes.add(size(child));
    }

    while (!childSizes.isEmpty()) {

        int smallest = childSizes.removeMin();

        if (smallest == -1 || smallest > currSize) {
            return -1; // Not a valid WQU
        }

        currSize += smallest;
    }

    return currSize;
}
```

Solution:

- (a) Iterate through the list of islands provided in the `isles` array. A negative number denotes the root of a disjoint set. If an island is the root, assign the `root` variable to it. Otherwise, locate its parent island using `isles[parent]`. You can add the current island as a child of its parent by doing `isles[parent].children.add(isles[i])`.
- (b) We iterate through the `isles` array of length N . Assigning a variable and adding to the back of a list are constant-time operations. Because there is also no early termination, this function runs in best- and worst-case $\Theta(N)$ time.
- (c) Tree 3 can be formed using the following union operations (in order):
1. (1, 4)
 2. (1, 5)
 3. (3, 6)
 4. (3, 7)
 5. (0, 2)
 6. (0, 8)
 7. (0, 1)
 8. (0, 3)
- (d) Proof by Contradiction: Say node B is connected to node A first. Then, node B is no longer and can never become the root of its subtree. Thus, it would be impossible for node C to be a direct child of node B because any union operation with node B will actually happen with its root, and not itself.
- (e) In this part, we will attempt to simulate a sequence of root-to-root union operations to create a given tree. Effectively, we are presenting a method such that if a tree is a valid WQU, it can be created using this sequence of operations. Solving this requires the following pieces of information:
1. First, note the observation made from part D. As a result, it becomes crucial that we attempt to construct this WQU from the bottom up. In other words, each subtree of a WQU must be fully constructed before we can union it to the new parent.
 2. Second, note that if a subtree could not be created, the entire tree is now invalid, regardless of ancestor. For example, tree 1 can never be created. Thus, even if it had other parents/ancestors, the entire tree is already invalidated.
 3. The next issue becomes: which subtree should you connect a parent root with first? Let's look at the trees given in part c. In Tree 2, Node 0 must be connected to three subtrees of sizes 1, 3, and 3.
- Node 0 on its own is initially a disjoint set of size 1. If you try unioning it with one of the larger subtree first, you'll notice it could not immediately become the parent (remember in a WQU, the root of the larger subtree becomes the parent of the smaller one). However, even if node 0 unions with node 2, it has a total size of 2, which is not sufficient to become the parent of the other subtrees.
4. However, Node 0 from Tree 3 has 4 child subtrees: sizes 1, 1, 3, and 3.

Once again, if you try unioning it with the larger subtrees, you would not obtain this structure. However, by first unioning with nodes 2 AND 8, node 0 can become the root of a size 3 subtree. Now, it can become the parent of nodes 1 and 3 through a union operation.

5. With some thought, this helps you conclude that the optimal strategy is to first union with the smaller subtrees. This allows a node to become the parent of larger and larger subtrees (as someone on staff mentioned, very much like agar.io).

We can start by filling in blanks 4-6. Because we know that we need the sizes of each subtree first, we can call the helper function on each child of a node, and add the result to `childSizes`. Since we observed that we should union with subtrees in sizes of ascending order, we can assign `childSizes` to a `PriorityQueue` in blanks 2-3.

Finally, we can fill out the while-loop. First, note that the `currSize` variable tells us the size of the current tree we are building. At the beginning, it is simply 1: the parent node. We want to continuously union a node with its assigned subtrees until there are none left, starting from the smallest.

We will pop the smallest subtree in line 8, and make the appropriate checks in lines 9 and 10. If one of our child subtrees cannot be created, then we know the entire tree is invalid. Furthermore, if the child subtree is larger than the current size of our tree, none of the other subtrees can be attached to it either. Thus, it is invalid. Finally, increment the size of the current subtree by the size of the smaller subtree. This is the new largest-size subtree we can take.

Finally, check in line 1 that the helper function does not return -1, meaning that the WQU tree can be successfully created.

4 Asymptotics

(16 points)

For each of the following statements, select whether you think the statement is always **True**, the statement is sometimes **False**, or that you **Decline** to answer.

Within each set of five, the following rubric will be used to score your answer:

- Each correct answer will give **+2** points.
- Each incorrect answer will give **-3** points.
- Answers marked “Decline” or left blank will be awarded **0** points.
- The maximum and minimum scores for each set of five is **8** points and **0** points, respectively.
- Example: 4 correct + 1 “Decline”. You receive +8 total points, which gets capped to 8/8.
- Example: 2 correct + 3 incorrect. You receive -5 total points, which gets capped to 0/8.

Set I (8 points)

Let f and g be real-valued, **positive**, strictly increasing functions.

- If $f(x) \in O(g(x))$, then $f(10^{100}) \leq g(10^{100})$.
- If $f(x)$ and $g(x)$ are the best- and worst-case runtimes of a program, respectively, then $f(x) \in O(g(x))$.
- For all functions $f(x)$, there exists some function $g(x) \neq f(x)$ such that $f(x) \in \Theta(g(x))$.
- If $f(x) \in \Theta(g(x))$, then $f^{-1}(x) \in \Theta(g^{-1}(x))$, where $f^{-1}(x)$ is the inverse function of f . (The inverse function of f is the unique function f^{-1} such that if $y = f(x)$, then $x = f^{-1}(y)$.)
- If a program uses $O(\log(N))$ memory in total (including inputs, local variables, and recursive function calls), then its worst-case runtime is $O(N^k)$ for some constant k .

- | | | |
|---|-------------------------------|--|
| (a) ☐ True | ☐ Decline | ☒ False |
| (b) ☒ True | ☐ Decline | ☐ False |
| (c) ☒ True | ☐ Decline | ☐ False |
| (d) ☐ True | ☐ Decline | ☒ False |
| (e) ☐ True | ☐ Decline | ☒ False |

Solution:

- O bounds only apply for sufficiently large values of x ; while 10^{100} is indeed a large value, it may not be sufficiently large for some pairs of functions. For example, consider $f(x) = 1000$, $g(x) = \log_{10}(x)$. Then $f(x) > g(x)$ for all $x < 10^{1000}$, even though eventually, $f(x)$ will become less than $g(x)$.
- By definition, this implies that $f(x) \leq g(x)$ for all x . Then $f(x) \leq 1g(x)$, so $f(x) \in O(g(x))$
- We can take $g(x) = 2f(x)$, which is always in the same Theta class as f . The only function where $2f(x) = f(x)$ is $f(x) = 0$, which is not a positive, strictly increasing function.
- Let $f(x) = \log_2(x)$, $g(x) = \log_3(x)$. Then $f^{-1}(x) = 2^x$, $g^{-1}(x) = 3^x$. f and g are in the same Theta class (they differ by a constant factor), but f^{-1} and g^{-1} are not (their ratio is 1.5^x , which grows to infinity)

- (e) The program `while(true) {pass;}` runs infinitely, but does not access more than constant memory. As an aside, this statement *is* true when we also require that the program terminates; see CS 172 for more info!

Set II (8 points)

- (a) $\sum_{i=1}^N i^2 \in \Theta(\sum_{i=1}^{N^2} i)$ (d) $\log(N) \in O(N^{0.0001})$
 (b) $\sum_{i=1}^N \log_{10}(i) \in \Omega(\sum_{i=1}^N \log_2(i))$ (e) $N! \in \Theta((N+1)!)$
 (c) $N^3 \in \Theta((N+1)^3)$

- | | | |
|---|-------------------------------|--|
| (a) ☐ True | ☐ Decline | ☒ False |
| (b) ☒ True | ☐ Decline | ☐ False |
| (c) ☒ True | ☐ Decline | ☐ False |
| (d) ☒ True | ☐ Decline | ☐ False |
| (e) ☐ True | ☐ Decline | ☒ False |

Solution:

- (a) The first sum evaluates to $\Theta(N^3)$, while the latter evaluates to $\Theta(N^4)$
 (b) $\log_2(x) = \log_2(10) * \log_{10}(x)$, so $\sum_{i=1}^N \log_2(i) = \log_2(10) * \sum_{i=1}^N \log_{10}(i)$
 (c) $(N+1)^3 = N^3 + 3N^2 + 3N + 1$, which is $\Theta(N^3)$ after ignoring lower-order terms
 (d) When N is doubled, $\log(N)$ increases by $\log(2)$, while $N^{0.0001}$ is multiplied by $2^{0.0001}$. For sufficiently large values of $N^{0.0001}$, multiplying by a small constant factor will yield a larger increase than adding a large constant factor.
 (e) The ratio of the two functions is $N+1$, which grows to infinity.

5 Hashing

(12 points)

As mentioned in class, putting mutable objects in a hash table is dangerous, even if the `hashCode` does not change when an object is mutated. It turns out that no matter what clever `hashCode` you try to devise, it is impossible to guarantee both efficiency and correctness when storing mutable objects.

For each of the following `hashCode`s for a `List<Integer>`, describe a flaw/limitation in the proposed design that will cause a `HashSet<List<Integer>>` to perform incorrectly or inefficiently. You should clearly state whether the issue with the `hashCode` is invalidity (the resulting hash table may yield incorrect behavior) or inefficiency (the hash table will work correctly, but may be slow).

If your answer is “invalidity”, give a sequence of operations that yields an incorrect result (either in code or clearly expressed in English). If your answer is “inefficiency”, briefly describe why. Answers with incorrect or missing explanations will be given zero credit.

Two `Lists` are considered equal if they have the same items in the same order.

- (a) (4 points) Use as a `hashCode` the memory address of the `List`, which doesn't change when the `List` gets mutated.

- (b) (4 points) Have `hashCode` always return 0.

- (c) (4 points) Create a new attribute `cachedHash` within the `List` class. When the `hashCode` method is first called on a `List`, compute the `hashCode` using `List<Integer>`'s normal `hashCode`, and save the result in `cachedHash`. After that, every future call of `hashCode` returns `cachedHash`, even if the list has been mutated.

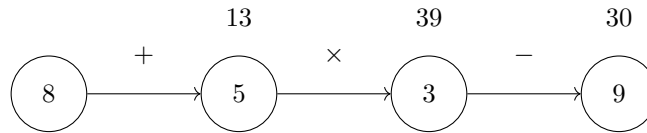
Solution:

- (a) Invalid. When putting two equal `Lists` into the `HashSet`, they may end up in different bins, because equal `Lists` can have different `hashCode`s
- (b) Inefficient. Since all `hashCode`s are equal, all items will be in the same bin, and we'll get linear runtime operations
- (c) Invalid. Insert the list `a = [1,2]` into the `HashSet`, then mutate it to `[1,2,3]`. Then try to insert the list `b = [1,2,3]` into the `HashSet`. Since the `hashCode` differs, there is a chance that this yields a `Set` with two copies of `[1,2,3]` in it.

6 Math Graphs

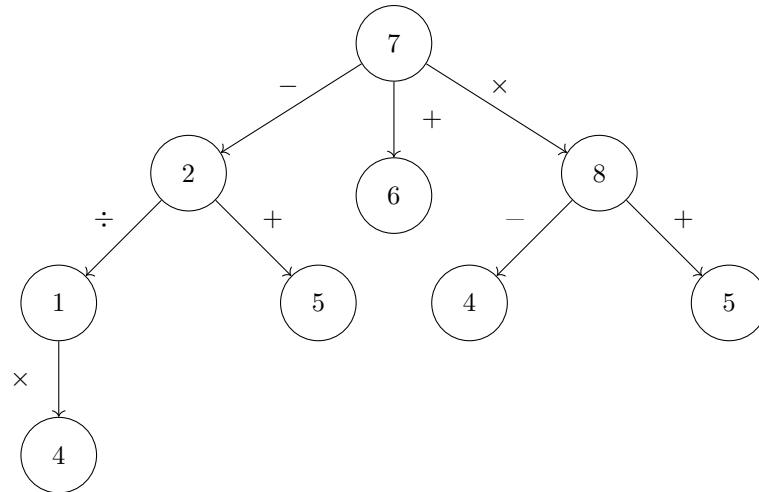
(11 points)

Aditya has created a type of graph to represent short arithmetic calculations called Math Graphs. In a Math Graph, each node contains a number, and each edge contains an arithmetic operation (addition, subtraction, multiplication, and floor division). A Math Path is a simple path in a Math Graph, and has a value equal to the result of computing the expression created along that path. For example:



The Math Path from 8 to 9 would have a value of $((8 + 5) \times 3) - 9 = 30$. Note that the value is computed in path order, and no operation has precedence over another.

(a) (1 point)



For the above Math Graph, what is the value of the largest-valued Math Path starting from 7? Write your answer as an integer.

Solution: 61. $7 \times 8 = 56 + 5 = 61$

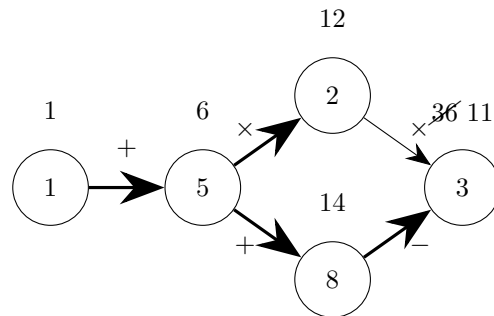
(b) (1 point) True/False: If a Math Graph is a rooted tree, then its largest-valued Math Path will start at the root and end at a leaf.

☐ True

☒ False

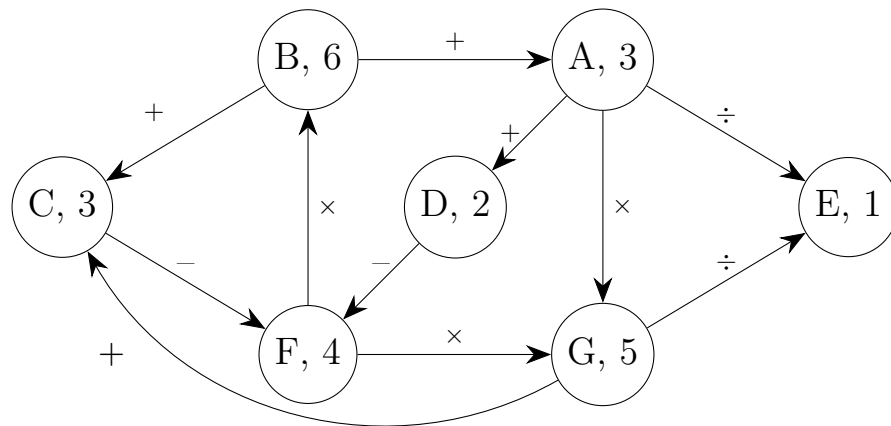
Solution: Take the above Math Graph, and add a “-10000000” node after each leaf.

Aditya wants to find the least-valued Math Path from one node to each other node of a Math Graph. He reasons that this is similar to finding the Shortest-Paths Tree of a graph, so he devises a slight modification to Dijkstra's algorithm. When evaluating a new path to a node, the appropriate math operation is performed between the current distance and the node's value instead of just adding an edge weight. For example:



When running Dijkstra's algorithm from node 1, we initially find a Math Path of value 36 to node 3. We later find a better path and relax it to a Math Path of value 11.

- (c) (6 points) Fill out the `distTo` and `edgeTo` arrays after running this modified version of Dijkstra's on the Math Graph below starting from node A:



Node	A	B	C	D	E	F	G
<code>distTo</code>	0						
<code>edgeTo</code>	-						

Solution:

Node	A	B	C	D	E	F	G
<code>distTo</code>	0	6	8	5	3	1	5
<code>edgeTo</code>	-	F	G	A	A	D	F

For each of the following types of Math Graph, determine if the algorithm above would **always** find a shortest paths tree (as defined previously). You may assume that all nodes are positive integers.

- (d) (1 point) A Math Graph with only addition (+) and multiplication ($\times$) operators.

☒ Always

☐ Not Always

- (e) (1 point) A Math Graph with NO division ($\div$) operators.

☐ Always☒ Not Always

(f) (1 point) A Math Graph that is a tree.

☒ Always☐ Not Always

Solution: Relaxation will only succeed when it shouldn't if it is possible to decrease distance by an action. Addition and multiplication will always increase (or at worst not decrease) the value of the path as long as the nodes have positive integers.

Even without division, subtraction will cause problems.

If the MathGraph is a tree, then the Shortest Path Tree is the only tree in existence, and therefore will necessarily be correct.

Nothing on this page is worth any points.

Plead the Fifth

(0 Points)

Write out the percentage of students who selected "Decline" on each part of Problem 4.

Feedback

(0 Points)

Leave any feedback, comments, concerns, or more drawings below!